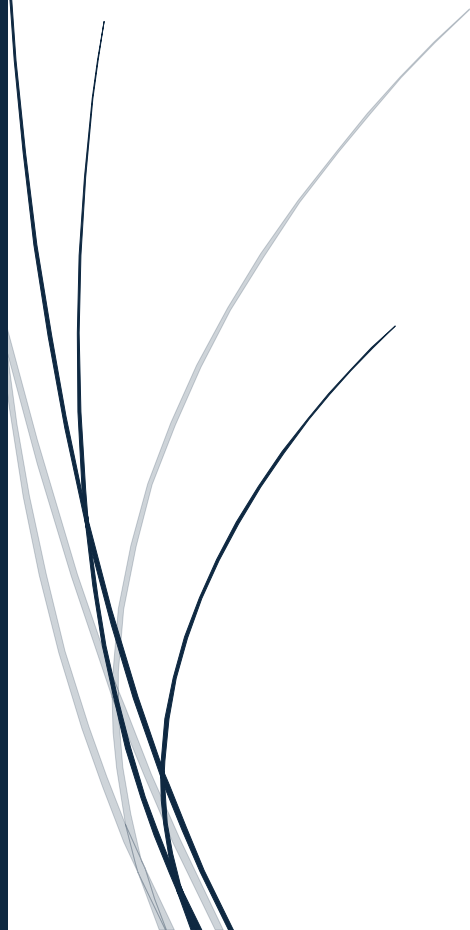


6-4-2025

DevOps

Eindopdracht Softwareontwerp &
-Architectuur 3



Baraa Bayrkdar
2210363

Inhoud

Inleiding	2
Functionele Requirements	2
Functionele Requirements	2
Niet-fuctionele Requirements	3
SMART Kwaliteitsdoelen	3
Architectuur & Ontwerp	4
Structuur Application Core:.....	4
Waarom deze structuur?.....	4
Diagrammen	5
4. Design Patterns	9
Waarom zijn deze patronen gekozen?	10
5. Teststrategie	11
Doelen van de teststrategie	11
Tools en technieken	11
Voorbeelden van geteste onderdelen	11
Kwaliteitsdoelen.....	12
6. CI/CD en Kwaliteitscontrole	13
Pipeline stappen:	13
SonarCloud-integratie	13
7. Conclusie en Reflectie	14
8. Bijlagen	15

Inleiding

Voor deze opdracht wordt een applicatie ontwikkeld die Scrum- en DevOps-projecten ondersteunt. Het systeem is geïnspireerd door bekende tools zoals Azure DevOps en Jira. Het doel is om een duidelijke, goed onderhoudbare en makkelijk testbare softwarekern te creëren. Hiervoor worden Clean Architecture en verschillende design patterns toegepast, die zorgen voor flexibiliteit en kwaliteit.

Functionele Requirements

In dit hoofdstuk worden de vereisten beschreven waaraan het systeem moet voldoen. Deze eisen zijn opgesteld volgens de SMART-methode: specifiek, meetbaar, acceptabel, realistisch en tijdgebonden.

Functionele Requirements

Dit zijn de belangrijkste functies die de applicatie moet bieden.

- Gebruikers kunnen binnen maximaal 5 seconden een nieuw Scrum-project aanmaken door het invoeren van een unieke naam, startdatum en minimaal één teamlid, gevalideerd met directe feedback vanuit het systeem.
- Gebruikers kunnen binnen maximaal 5 seconden backlog-items aanmaken, wijzigen, verwijderen en koppelen aan exact één sprint en precies één developer via een duidelijke gebruikersactie met onmiddellijke systeemvalidatie.
- Gebruikers kunnen binnen maximaal 5 seconden ten minste één activiteit toevoegen aan een backlog-item om taken specifiek onder meerdere developers te verdelen, waarbij iedere activiteit aan exact één developer gekoppeld wordt.
- Backlog-items wijzigen uitsluitend in de volgorde: todo → doing → ready for testing → testing → tested → done. Elke afwijking wordt direct geblokkeerd door het systeem.
- Bij elke statuswijziging van een backlog-item verstuurt het systeem binnen maximaal 2 seconden automatisch een notificatie via e-mail of Slack naar alle betrokken gebruikers, met logging voor verificatie.
- Gebruikers kunnen discussies starten en reacties plaatsen op backlog-items zolang het item niet de status 'done' heeft bereikt. Na het bereiken van de status 'done' wordt verdere discussie direct geblokkeerd door het systeem.
- De scrum master kan binnen maximaal 5 seconden sprints beheren (aanmaken, starten en afronden). Indien de sprint is afgerond, start binnen maximaal 2 seconden automatisch een gekoppelde development-pipeline, mits deze geconfigureerd is.

- Development-pipelines starten automatisch binnen maximaal 2 seconden na afronding van een release-sprint. Resultaten van de pipeline worden binnen 10 seconden gerapporteerd aan de scrum master via notificatie en logbestanden.
- De scrum master kan per sprint binnen maximaal 5 seconden een rapport genereren met minimaal voortgangsindicatoren en teaminformatie in PDF- of PNG-formaat, inclusief bedrijfsnaam en datum in header/footer.

Niet-fuctionele Requirements

Deze eisen beschrijven de kwaliteitseigenschappen van het systeem.

- De broncode bereikt uiterlijk bij oplevering een Quality Gate A-rating in SonarCloud, wat inhoudt: minder dan 3% duplicatie, geen kritieke bugs en maximaal 5 code smells per 1.000 regels code.
- Minimaal 90% van alle domeinlogica wordt bij oplevering aantoonbaar gedekt door automatisch uitgevoerde unit tests.
- Het systeem is modulair en uitbreidbaar volgens het Open-Closed principe, gedocumenteerd in het ontwerp met minimaal twee praktijkvoorbeelden voor toekomstige uitbreidingen die zonder bestaande codewijzigingen gerealiseerd kunnen worden.
- Statuswijzigingen van backlog-items worden inclusief validatie en notificatie altijd binnen maximaal 3 seconden verwerkt en bevestigd aan gebruikers via logging en notificatiebevestiging.
- 100% van de businesslogica is testbaar zonder afhankelijkheid van infrastructuurcomponenten zoals databases of externe services, bewezen via geïsoleerde unit tests bij oplevering.

SMART Kwaliteitsdoelen

- **Code Coverage:** Bij oplevering is minimaal 90% van de code aantoonbaar gedekt door unit tests.
- **Duplicatie:** Bij oplevering is minder dan 3% code-duplicatie meetbaar in SonarCloud.
- **Maintainability:** De code haalt uiterlijk bij oplevering een Maintainability Rating A volgens SonarCloud (duidelijk meetbaar in rapportage).
- **Reliability & Security:** De code heeft uiterlijk bij oplevering een Reliability & Security Rating A, wat betekent: geen kritieke bugs en geen kwetsbaarheden volgens SonarCloud.

Architectuur & Ontwerp

De architectuur van het systeem is gebaseerd op de Clean Architecture, die zorgt voor duidelijke scheiding en onafhankelijkheid van systeemcomponenten. Dit betekent dat de belangrijkste logica van het systeem netjes gescheiden is van technische onderdelen. Deze opbouw maakt het systeem duidelijk, goed testbaar en makkelijk uit te breiden.

Structuur Application Core:

De applicatie bestaat uit de volgende lagen:

- **Domeinlaag**
Deze laag bevat de kern van het systeem, zoals Project, Sprint, BacklogItem, Comment en Task. Hier zitten de regels en afspraken van het systeem, onafhankelijk van externe techniek.
- **Applicatielaag**
Deze laag zorgt ervoor dat de onderdelen uit de domeinlaag goed samenwerken. Hier zit bijvoorbeeld de logica voor het starten of afronden van een sprint, of het wijzigen van een status.
- **Interface-laag**
Hier worden afspraken (interfaces) gedefinieerd voor onderdelen die met de buitenwereld praten, zoals notificaties of data-opslag.
- **Infrastructuurlaag**
Dit is de buitenste laag waar de echte verbindingen met bijvoorbeeld e-mail, Slack of Git gemaakt worden. Deze laag past zich aan op basis van wat de binnenste lagen nodig hebben.

Volgens de Onion architectuur weten de binnenste lagen niets van de buitenste lagen. Dit zorgt voor duidelijke scheiding en voorkomt afhankelijkheid van techniek.

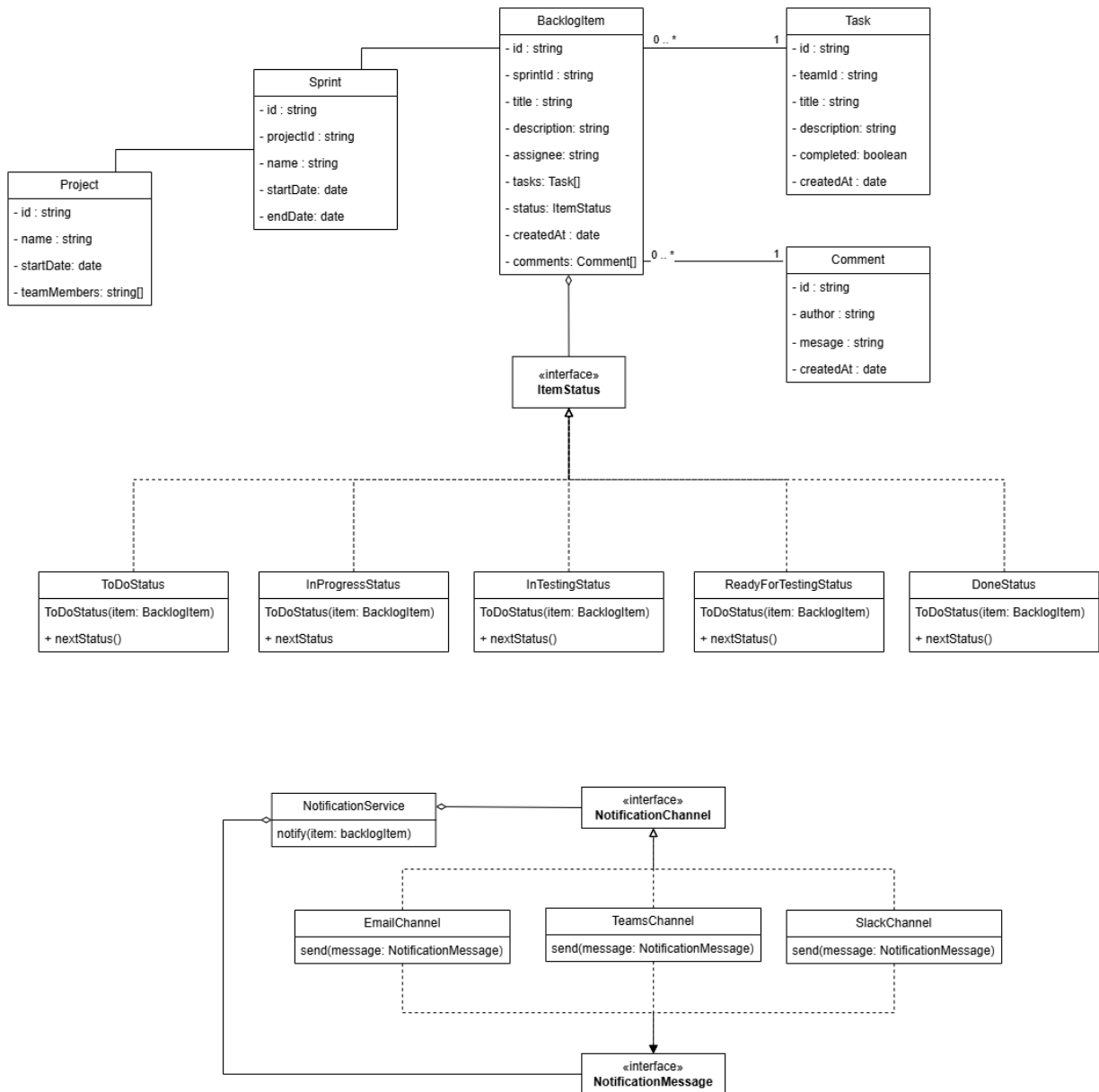
Waarom deze structuur?

- **Betere testbaarheid:** De belangrijkste logica is makkelijk te testen zonder externe systemen.
- **Uitbreidbaar:** Nieuwe functies kunnen worden toegevoegd via interfaces zonder bestaande code te breken.
- **Duidelijkheid:** Elke laag heeft een duidelijke taak.
- **Betrouwbaarheid:** Door vaste patronen te gebruiken is de code minder foutgevoelig.

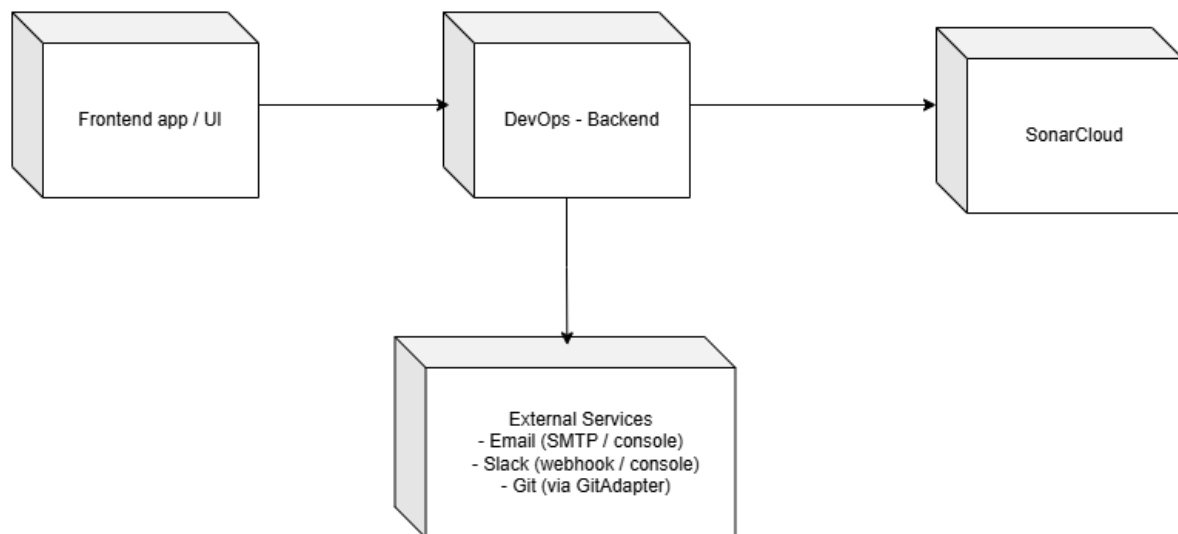
Diagrammen

Om de structuur visueel duidelijk te maken, zijn de volgende diagrammen toegevoegd:

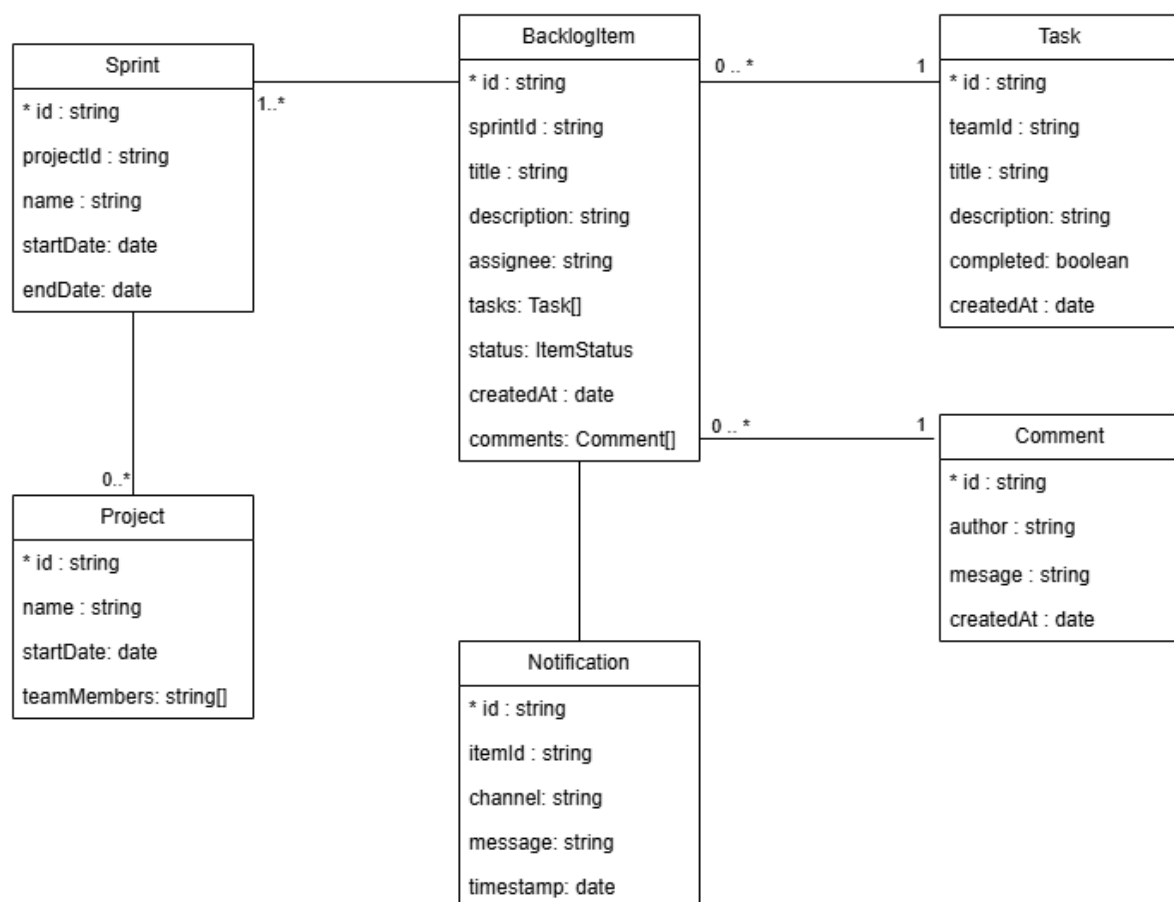
- Class-diagrammen: tonen de structuur van de belangrijkste klassen.



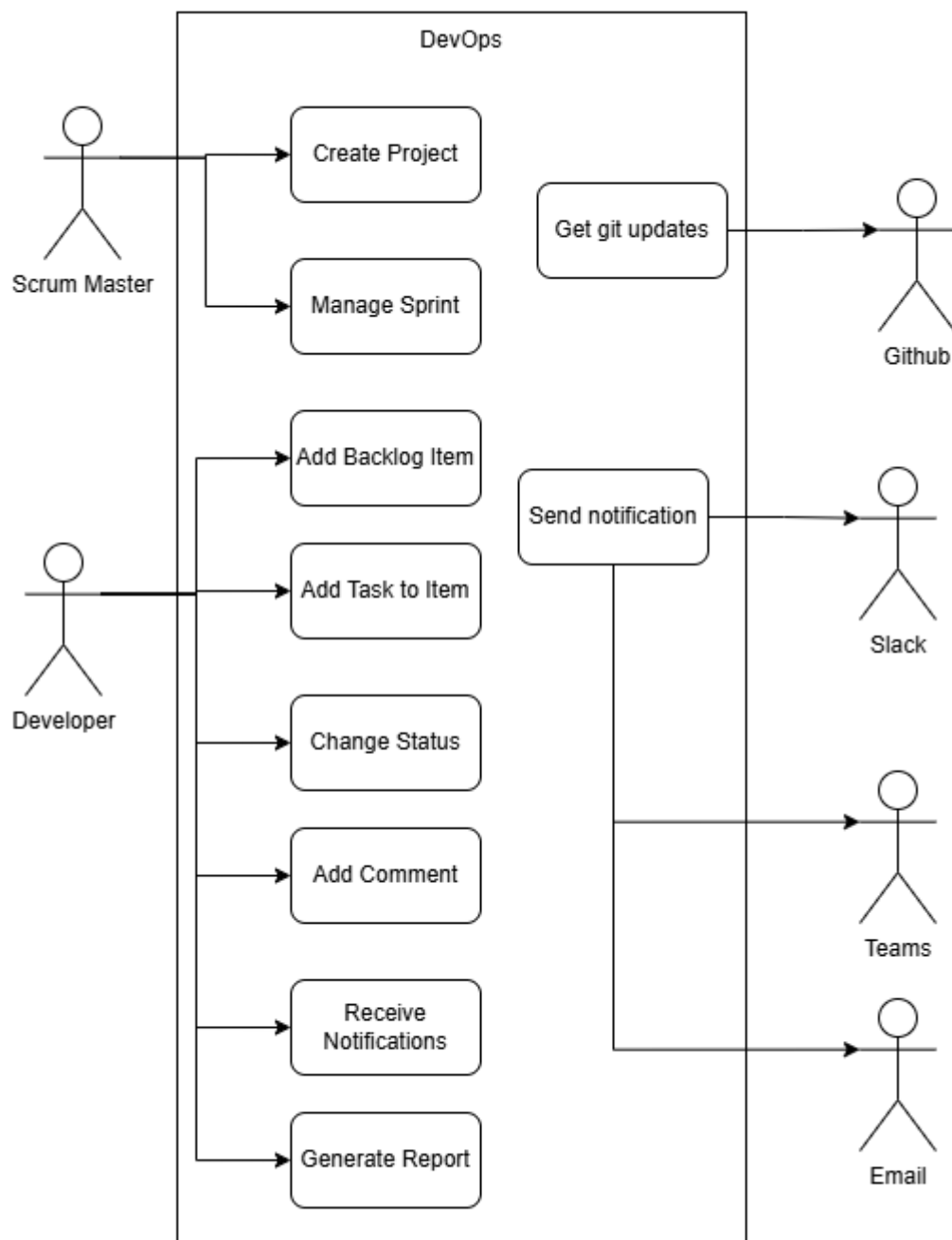
- fysiek-diagram: laat het nodes en interactie van systemen zien.



- ERD diagram:



- UseCase Diagram:



- Lagenoverzicht: laat zien hoe de afhankelijkheden tussen lagen zijn opgebouwd.

```
project-root/  
├─ domain/  
│  ├─ models/  
│  └─ statuses/  
├─ services/  
├─ interfaces/  
├─ infrastructure/  
├─ adapters/  
├─ visitors/  
├─ tests/  
└─ main.ts
```

4. Design Patterns

Om het systeem flexibel, onderhoudbaar en duidelijk gestructureerd te houden, zijn er verschillende **design patterns** toegepast. Deze patronen zorgen ervoor dat de code begrijpelijk blijft en eenvoudig kan worden uitgebreid zonder bestaande onderdelen aan te passen.

Hieronder volgt een overzicht van de gebruikte patronen, met per pattern een korte toelichting en de manier waarop het is toegepast in deze applicatie:

Pattern	Toepassing in de applicatie
Strategy	Voor het verzenden van notificaties worden verschillende strategieën gebruikt, zoals e-mail, Teams en Slack. Elk kanaal implementeert een interface, waardoor eenvoudig nieuwe kanalen kunnen worden toegevoegd.
Observer	Wanneer de status van een backlog-item wijzigt, worden automatisch notificaties verstuurd naar betrokken gebruikers. Dit gebeurt zonder dat het item zelf hoeft te weten wie er precies op de hoogte moet worden gebracht.
State	De status van een backlog-item verloopt via vaste stappen (zoals 'To Do' → 'In Progress' → ...). Iedere status weet wat de volgende mogelijke stap is, waardoor het proces gecontroleerd en foutloos verloopt.
Template	Voor het uitvoeren van vaste processen, zoals het afronden van een sprint of het voorbereiden van een release, wordt een standaard sjabloon gebruikt. Hierdoor is de structuur van het proces herbruikbaar en consistent.
Adapter	Om integratie met externe systemen zoals Git mogelijk te maken, wordt een adapter gebruikt. Deze vertaalt de interne structuur van de applicatie naar het formaat dat Git verwacht, zonder dat de interne logica hoeft te worden aangepast.
Composite	Een backlog-item kan bestaan uit meerdere onderliggende taken. Dit is geïmplementeerd als een boomstructuur waarin zowel het item als de taken op dezelfde manier behandeld kunnen worden.
Visitor	Voor het genereren van rapportages (zoals tekst, CSV of JSON) wordt het visitor pattern gebruikt. Hierdoor kunnen verschillende rapportformaten worden toegevoegd zonder de onderliggende modellen te veranderen.

Waarom zijn deze patronen gekozen?

- **Herbruikbaarheid:** Dezelfde patronen kunnen in meerdere onderdelen van de applicatie terugkomen.
- **Scheidingsprincipes:** Ieder onderdeel heeft één duidelijke verantwoordelijkheid.
- **Uitbreidbaarheid:** Nieuwe functies kunnen worden toegevoegd door nieuwe klassen toe te voegen in plaats van bestaande code te wijzigen.
- **Testbaarheid:** Patronen zoals Strategy en Adapter maken het eenvoudig om onderdelen los te testen met mocks of alternatieve implementaties.

5. Teststrategie

Om de kwaliteit en betrouwbaarheid van het systeem te waarborgen, is er een duidelijke teststrategie toegepast. Hierbij is de nadruk gelegd op het testen van de **services** en de **rapportagefunctionaliteit**. Dankzij de structuur van Clean Architecture is de belangrijkste logica eenvoudig testbaar zonder afhankelijkheid van infrastructuur zoals een database of externe APIs.

Doelen van de teststrategie

- Verifiëren dat de **bedrijfsregels correct worden uitgevoerd**.
- Garanderen dat **statusveranderingen** van backlog-items juist verlopen.
- Controleren of bij bepaalde gebeurtenissen automatisch de juiste **notificaties** worden verstuurd.
- Testen of rapportages correct worden gegenereerd op basis van ingevoerde gegevens.
- Aantonen dat de kernlogica **geïsoleerd en betrouwbaar** werkt.

Tools en technieken

- **Testframework: Jest** is gebruikt voor het schrijven en uitvoeren van tests.
- **Mocking:** Via Jest-mocks zijn afhankelijkheden zoals notificaties of repositories nagebootst.
- **Code coverage:** De testdekking wordt automatisch gemeten, met als doel minimaal **90%** dekking van alle domeinlogica.

Voorbeelden van geteste onderdelen

Testscenario	Beschrijving
Statusovergang To Do → Done	Controleert of statuswijzigingen alleen via geldige paden verlopen.
Verwerking van notificaties	Test of alle betrokken kanalen een bericht ontvangen na statuswijziging.
Blokkade van discussies bij Done	Controleert of discussies niet meer mogelijk zijn na afronding van een item.
Rapportage met verschillende visitors	Test of bezoekers (CSV, JSON, Markdown) correcte output genereren.
Sprint afronden zonder voltooide items	Controleert of er waarschuwingen worden gelogd voor onafgemaakte items.

Kwaliteitsdoelen

De volgende kwaliteitsdoelen zijn onderdeel van de teststrategie:

- **Code Coverage $\geq 90\%$:** Aangetoond via geautomatiseerde rapporten.
- **Geen kritieke bugs:** Geen rode flags in SonarCloud op het moment van oplevering.
- **Maintainability Rating A:** De code is goed gestructureerd en makkelijk aanpasbaar.
- **Testbaar zonder infrastructuur:** Alle belangrijke logica is los te testen via mocks en stubs.
-

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	92.45	50	88.23	92.45	
domain/models	86.36	50	80	86.36	
BacklogItem.ts	81.25	0	75	81.25	29-32
Task.ts	100	100	100	100	
domain/statuses	95.83	100	90	95.83	
DoneStatus.ts	75	100	50	75	8
InProgressStatus.ts	100	100	100	100	
InTestingStatus.ts	100	100	100	100	
ReadyForTestingStatus.ts	100	100	100	100	
ToDoStatus.ts	100	100	100	100	
services	100	100	100	100	
NotificationService.ts	100	100	100	100	
StatusTransitionManager.ts	100	100	100	100	

Figuur 1 Report of testing using Jest and SonarCloud

6. CI/CD en Kwaliteitscontrole

Om kwaliteit, snelheid en betrouwbaarheid in het ontwikkelproces te garanderen, is een CI/CD-pipeline opgezet met GitHub Actions. Deze pipeline zorgt ervoor dat elke wijziging in de code automatisch wordt gecontroleerd, getest en beoordeeld op kwaliteit.

De integratie met SonarCloud is hierbij essentieel voor het bewaken van codekwaliteit, duplicatie, security en testdekking.

Pipeline stappen:

Bij elke push of pull request naar de hoofdbbranch wordt automatisch de volgende workflow uitgevoerd:

- **Installeren van dependencies**
De applicatie installeert alle benodigde pakketten via npm ci.
- **Uitvoeren van unit tests**
Alle tests worden uitgevoerd met Jest. Bij fouten stopt de pipeline.
- **Metten van code coverage**
De dekking van de tests wordt automatisch gecontroleerd. Het doel is $\geq 90\%$.
- **Analyse door SonarCloud**
De code wordt doorgelicht op duplicatie, code smells, bugs en vulnerabilities. De projectstatus moet voldoen aan Quality Gate A.
- **Releasevoorbereiding**
Bij het afronden van een sprint kan automatisch een nieuwe Git-tag worden voorbereid, klaar voor een (simulatie van) release.

SonarCloud-integratie

De integratie met SonarCloud biedt realtime inzicht in:

- **Code duplicatie** (doel: $<3\%$)
- **Bugdetectie en security-issues**
- **Maintainability index**
- **Code coverage**
- **Reliability & security rating** (doel: A)

Het project voldoet bij oplevering aan de **volledige Quality Gate A**-eisen van SonarCloud.

7. Conclusie en Reflectie

In dit project is een basis gebouwd voor een DevOps/Scrum-applicatie waarmee je projecten, sprints, taken en backlog-items kunt beheren. Bij het ontwerpen van het systeem lag de focus op duidelijkheid, onderhoudbaarheid en goede testbaarheid. Door gebruik te maken van **Clean Architecture** zijn de verschillende onderdelen van het systeem netjes van elkaar gescheiden. Dat maakt het makkelijk om het systeem in de toekomst uit te breiden.

Er zijn verschillende **ontwerpprincipes** (design patterns) gebruikt, zoals Strategy, State en Visitor. Deze zorgen ervoor dat nieuwe functies toegevoegd kunnen worden zonder bestaande code te hoeven aanpassen. Daarnaast zijn er veel **unit tests** geschreven met Jest om zeker te weten dat de belangrijkste onderdelen goed werken. Omdat de logica losstaat van technische systemen zoals databases, konden de tests makkelijk en betrouwbaar worden uitgevoerd.

De integratie met **SonarCloud** en **GitHub Actions** zorgt ervoor dat de code automatisch gecontroleerd wordt bij elke wijziging. Hierdoor blijft de kwaliteit van de code hoog en worden fouten snel gevonden. Het systeem voldoet aan de gestelde kwaliteitsdoelen, zoals een testdekking van minstens 90% en een goede beoordeling in SonarCloud.

Samengevat is er een sterke en goed georganiseerde softwarekern opgeleverd die werkt zoals bedoeld. Het systeem is klaar om verder uitgebreid te worden, bijvoorbeeld met gebruikersrollen, autorisatie of een visuele gebruikersinterface.

8. Bijlagen

- UML-diagrammen
- Testcases
- SonarCloud-rapport
- Broncode